Lecture III - Packages and Data Management

Applied Optimization with Julia

Dr. Tobias Vlćek
University of Hamburg

Recap of Last Week

Goals for Today

After this lecture, you will:

- Have **refreshed the basics** from last week's tutorials
- Have seen Git again, **nice and slow**
- Know what packages are and why we use them
- Have seen your first DataFrame
- Be ready to start this week's tutorials

Variables and Data Types

- Variables are used to store values
- Assign a value to a variable using the `=` operator
- You can use **different data types** for variables
- You can change the value of a variable



You can use the `typeof` function to check the type of a variable.

Vectors and Matrices

- Vectors and matrices are used to store multiple values
- You can create them using the `[` and `]` operators
- Access their elements using square brackets



You can use the `push!` function to add elements to a vector or the `pop!` function to remove elements from a vector.

Comparisons and Logic

- Comparisons are **used to compare values**
- `==` checks if two values are equal
- `!=` checks if two values are not equal
- `<` and `>` check if one value is smaller or greater than the other
- `<=` and `>=` also allow the two values to be equal
- `&&` is true if **both** conditions are true
- `||` is true if **at least one** of two conditions is true

Loops

- Loops are used to repeat code
- `for` loop repeats code for a fixed number of times
- `while` loop repeats code as long as a condition is true

```
1 for product in ["apple", "banana", "cherry"]
2     println("We sell: $product")
3 end
```

```
We sell: apple
We sell: banana
We sell: cherry
```

Conditionals

- Conditionals run code **only under certain conditions**
- `if` executes code if a condition is true
- `elseif` checks a further condition if the previous one was false
- `else` executes code if none of the conditions were true

> ⓘ Conditionals are not loops: they decide **whether** code runs, while loops decide **how often** it runs.

Dictionaries

- Dictionaries store **key-value pairs**, like a lookup table
- Access a value by its key using square brackets
- `keys` and `values` return all keys and values

```
1 student_ids = Dict("Elio" => 1001, "Bob" => 1002)
2 println("Elio's ID: ", student_ids["Elio"])
```

```
Elio's ID: 1001
```



Later in the course, we will use dictionaries to store and look up the data of our optimization models.

Git, One More Time

Last week's Git part was **a lot at once**. Here is the short version:

- Git takes **snapshots (commits)** of your project, so you can always go back
- The everyday workflow is just two steps in Zed's Git panel:
  1. **Stage** your changed files
  2. Write a short message and **commit**

> Commit after each tutorial exercise, that is a good rhythm. History and undoing can wait.

> No Git setup yet? No problem: do it this week, and ask me if anything blocks you.

Git and Your Group

- You hand in the assignments as a **group of up to three**. So why not share one repository?
- One person creates a repository on GitHub and invites the others
- Everyone works on their own copy and shares changes via **push** and **pull**
- You always see **who changed what, and when**

> ⓘ This is optional: a shared cloud folder works too. But a shared repository is exactly how teams share code in practice.

Packages & Data

Why Packages?

Question: Do we have to write all code ourselves?

- Luckily not! We can use **packages** written by others
- Packages are collections of reusable code, e.g. `DataFrames.jl` for tables or `Plots.jl` for charts
- Julia's package manager `Pkg` installs them for us:

```
1 using Pkg
2 Pkg.add("DataFrames")
```

Data Management with DataFrames

- `DataFrames.jl` stores tabular data, like a **spreadsheet in code**

```
using DataFrames
sales = DataFrame(
    product = ["apple", "banana", "cherry"],
    price = [0.50, 0.30, 2.00],
    sold = [120, 90, 30]
)
```

- This week's tutorials show how to read, write, and plot such data

Data Management with DataFrames

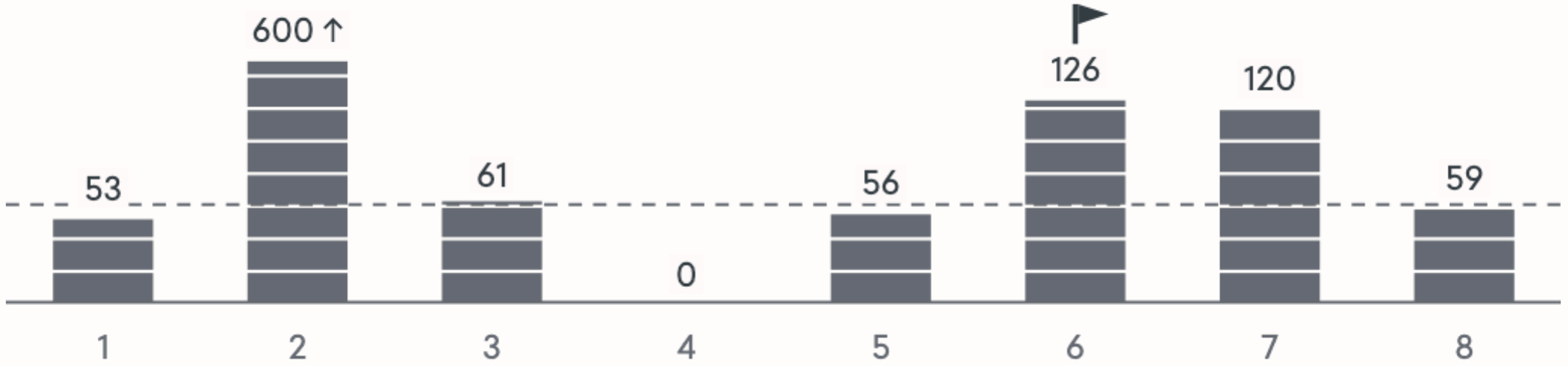
3×3 DataFrame

Row	product	price	sold
	String	Float64	Int64
1	apple	0.5	120
2	banana	0.3	90
3	cherry	2.0	30

Your Turn

The brewing plan comes straight from the sales team's order file, in crates of 20 bottles, and the file was imported with errors. Tap a week you don't trust to replace its order with the median, so that no crate is short or left over. A flag marks the harbour festival.

Orders in crates, dashed: median 60



Brewing plan in crates



Short (–) or left over (+)

+540

You

540 crates wrong

Ready: press Optimize to compare with the best plan.

Optimize

Reset

New

Five Tutorials

Topics of the Tutorials

- Functions: Learn how to define and use functions
- Packages: Learn how to install and use packages
- DataFrames: Learn how to work with tabular data in Julia
- IO: Learn how to read and write data in Julia
- Plots: Learn how to create plots in Julia

Get Started with the Tutorials

- Download this week's tutorials and start with the first one
- **Remember, you can ask questions anytime in class or via email!**

> ⓘ And that's it for this lecture!
>
> The remaining time we will already start working on this week's tutorials.

Literature

Literature

- Lauwens, B., & Downey, A. B. (2019). Think Julia: How to think like a computer scientist (First edition). O'Reilly®. Link to the free book website.
- Julia Documentation

For more interesting literature to learn more about Julia, take a look at the literature list of this course.